

Geometry- and Physics-Guided IR Train Proof Contract

Proof repair artifact for Immersive Railroading 1.11.0

2026-08-16

This artifact replaces the PID proof. Its honest status is **NOT_READY**. Lean proves the exact-rational contract lemmas listed below, but the complete runtime refinement from Lua, Java floating point, actual track samples, and the full multi-particle IR plant is still an implementation obligation.

Contents

1	Scope and claim discipline	1
2	Provenance	2
3	Geometry: traced centreline to spline	2
3.1	Cubic representation	2
3.2	Tracer reference and residual	2
4	Graph, markers, and route locality	3
5	Detector profile	3
6	Directed stopping coordinate	3
7	IR physics derivation	4
7.1	Material, efficiency, and pressure branches	4
7.2	Signed equation	4
8	Combined command and plant transition	5
9	Uniform guard and safety envelope	5
10	Arrival, holding, and failure states	5
11	Claim inventory	6
12	Implementation funnel	6
13	Readiness	7

1 Scope and claim discipline

The proof target is robust front-coupler arrival at a directed station or signal, within a derived computational tolerance. The representation pipeline is deliberately not a PID pipeline:

tracer samples $\longrightarrow$ track-centre cubic splines $\longrightarrow$ directed routing graph $\longrightarrow$ detected consist profile
 $\longrightarrow$ front-coupler controller $\longrightarrow$ IR plant.

The spline model is geometric source of truth. The graph is a routing abstraction. The graph never edits or fits a spline. A station marker is not an arbitrary graph vertex, and a raw entity position is not silently treated as the rail centreline or the front coupler.

Every result in this document has one of these classifications:

- **bytecode fact**: directly observed in the Java-8 root class or manifest;
- **derived from bytecode**: an algebraic consequence of those observations;
- **certified model assumption**: an explicit assumption checked at the runtime boundary;
- **runtime contract**: a required data, timing, or fail-closed contract;
- **future Lua implementation obligation**: a precise future production-Lua change;
- **unsupported or withdrawn**: intentionally not claimed.

2 Provenance

The authoritative input is the extracted tree under `immersive_railroading/.cache/jar`. The manifest reports `Multi-Release: true`; the root class files have major version 52 (Java 8), while the classes under `META-INF/versions/16` have major version 60. The root Java-8 classes are the target; version-16 classes are not used silently. The mod metadata reports Minecraft 1.7.10 and Immersive Railroading 1.11.0.

The exact manifest, class hashes, and decompiler-input hashes are recorded in `pid-jar-manifest.md` and `pid-bytecode-traceability.md`. The proof does not use a historical PID theorem as authority. The old files were inspected only after this model and bytecode pass, to identify the migration surface and obsolete claims.

3 Geometry: traced centreline to spline

3.1 Cubic representation

A physical section is a 3D cubic Bezier curve

$$B(t) = (1-t)^3 p_1 + 3(1-t)^2 t p_2 + 3(1-t) t^2 p_3 + t^3 p_4, \quad 0 \leq t \leq 1.$$

The Lean structure `Cubic` stores the four 3D points and `cubicPosition` and `cubicDerivative` use the Bernstein expressions. The reverse orientation is the control-point reversal `cubicReverse`. The endpoint and derivative facts are proved by Lean. The derivative endpoint is $3(p_2 - p_1)$ and $3(p_4 - p_3)$, not an unscaled chord.

Straight track is encoded as a degenerate cubic with both interior controls at the endpoint displacement. This is a representation choice, not a claim that all IR curves are straight. The Java root implementation of `CubicCurve.circle()` uses a cubic approximation constant `0.55191502449`; therefore IR turns are not described as exact circles. Arc length is stored by certified marks or a certified numerical approximation. Each spline stores tangent, grade, curvature, fitting error, trace identifiers, optional builder metadata, and a stable identifier.

The geometric certificate requires monotone (t, s) arc marks, positive total length, nonnegative fit error, correct reverse controls, and tangent metadata. It is implemented by `SplineCertificate`. Curvature and grade are metadata/derived quantities and must be bounded before they enter a stopping certificate.

3.2 Tracer reference and residual

The tracer record contains tick, raw IR position, speed, heading, tracer definition ID, stock UUID, validity, and sample age. A tracer configuration contains a reference offset, sampling spacing, curvature bound, floating-point error, and missing-observation bound. The centre estimate is

$$c_i = r_i + o_{\text{tracer}},$$

and the certified residual budget is

$$\varepsilon_{\text{trace}} = \Delta \ell \kappa_{\text{max}} + \varepsilon_{\text{float}} + \varepsilon_{\text{missing}}.$$

The proof record additionally requires a residual vector e_i with

$$c_i^{\text{actual}} = c_i + e_i, \quad \|e_i\|_1 \leq \varepsilon_{\text{trace}}.$$

This is a certified model assumption until the Lua sampling loop and the exact stock/bogey offsets are implemented and measured. It explicitly avoids the invalid claims that raw `getPos()` is the F3 hitbox or the exact rail centreline, and that the tracer reference is the front coupler.

Repeated traces are matched in 3D using distance, tangent, curvature, and arc coordinate. A match can extend, continue, overlap, split, intersect, or create new geometry. Two projected paths that cross in 2D are not merged. The automatic topology classifier has the following conservative policy:

Observation	Routing result
same interval	continuation
interval extension	extension
validated physical junction	shared connection
both paths continue at same level without connection	isolated crossing
3D height separation above tolerance	overpass
insufficient or contradictory observation	unresolved

Only continuation, extension, and validated shared connections are routable. Crossings preserve independent spline paths. An overpass is separate when its 3D separation exceeds the certified tolerance. Unexplored branches, coincident same-level paths with indistinguishable connectivity, and hidden run-time connectivity remain unresolved and unavailable for routing; manual classification is required before activation.

4 Graph, markers, and route locality

The routing graph stores spline IDs and arc-length intervals on edges. Vertices come from spline endpoints, validated junctions, approved switch/merger nodes, and marker mappings. A physical spline normally produces symmetric forward and reverse edges. Direction restrictions are imposed on the relevant edge or interval, not by changing the spline geometry.

A station or signal is mapped to the closest spline only when its 3D distance is at most its explicit tolerance. The mapping stores marker ID, spline ID, arc coordinate, measured distance, kind, and exactly one configured approach orientation. A marker too far from every spline is rejected. The opposite approach is not accepted unless separately configured.

An active route stores graph revision, spline IDs, vertex IDs, and edge IDs. Validation checks only those referenced objects and the current graph revision; it does not scan the entire graph. A revision change invalidates the route, invokes route finding against the latest graph, and enters fail-closed stop with an in-game report when no route exists. Geometry reconstruction and dynamic occupancy/routing are separate operations.

5 Detector profile

The detector contract consumes the `ir_train_overhead` event and records its `stock_uuid`. The detector's `info()` call is immediate and records definition ID, weight, direction, speed, brake channels, brake-system and adhesion data, and locomotive data when present. Catalogue order is the event order, including repeated definition IDs. `consist()` is used only as aggregate validation; it does not replace per-stock records.

The initial catalogue speed is no greater than approximately 10 km/h. A higher speed is permitted only after a detector-length, minimum-spacing, event latency, and `info()` response-time bound is measured. The catalogue profile is immutable during a run. A UUID/order/definition/weight or brake mismatch reports an in-game error, stops the controller, and requires player recataloguing.

6 Directed stopping coordinate

For the selected directed spline orientation, s increases along the active route. The front coupler, derived from stock definition and IR coupler semantics, is the stopping reference:

$$x = s_{\text{target}} - s_{\text{front coupler}}, \quad v_s = \frac{ds_{\text{front coupler}}}{dt}.$$

The theorem is only entered under $x > 0$ and $v_s > 0$. Wrong-way motion, away-motion, target crossing, unavailable orientation, invalid localization, or stale data enters recovery or fail-closed state. World direction is handled by selecting the appropriate directed spline orientation; it is not handled by silently accepting an incorrectly oriented marker.

7 IR physics derivation

This section records the equation used by Lean and the controller contract. The evidence and exact bytecode locations are in the traceability file.

7.1 Material, efficiency, and pressure branches

The Java-8 root constants are:

$$\mu_{ss}^{static} = 0.70 = \frac{7}{10}, \quad \mu_{ss}^{kinetic} = 0.42 = \frac{21}{50}, \quad \mu_{sci}^{kinetic} = 0.25 = \frac{1}{4}, \quad g = 9.8 = \frac{49}{5}.$$

The effective pressure is the maximum of train-brake and independent-brake position, normalized to the configured range. In the contract model it is

$$p = \min(1, \max(p_{train}, p_{ind})).$$

This explicitly retains both brake channels. Normalization rejects NaN and clamps values outside $[-1, 1]$ at the API boundary; invalid or unavailable values are a runtime failure, not a mathematical zero.

For a physics input record z , the ordinary brake demand is

$$F_{brake, design} = m_{design} g \mu_{sci}^{kinetic} p,$$

then the wheel-slip branch replaces it with the steel kinetic term

$$F_{brake, slip} = m g \mu_{ss}^{kinetic}$$

when the root locomotive branch reports slipping and $|v| > 0.01$ m/s. The configured `brakeMultiplier` multiplies the selected branch. The maximum adhesion term uses current mass, steel static friction, gravity, and the separate brake-adhesion efficiency. Cogging locomotives override both brake-system and brake-adhesion efficiencies to 10 in the inspected root override; ordinary rolling stock uses its configured brake-shoe friction and base adhesion efficiency.

7.2 Signed equation

The signed route force is derived in this order, preserving signs and units:

$$\begin{aligned} F_{grade} &= m(-g) \sin(\theta) M_{slope}, \\ F_{traction} &= u_{throttle} F_{available}, \\ F_{rdi} &= F_{rolling} + F_{direct} + F_{interference} + F_{near-zero}, \\ F_{coupler} &= F_{curvature} + F_{slack} + F_{push/pull}, \\ F_{net} &= F_{grade} + F_{traction} - F_{rdi} - F_{brake} - F_{coupler}, \\ a &= F_{net}/m. \end{aligned}$$

The Lean definitions `gradeForceN`, `tractiveForceN`, `rollingDirectInterferenceN`, `couplerAdverseN`, `brakeForceN`, and `signedNetForceN` are connected by the proved `signed_force_equation` identity. The force ledger is not an unexplained abstraction: each summand corresponds to one root bytecode branch.

The root `SimulationState.forcesNewtons()` contributes grade and tractive force. `frictionNewtons()` contributes rolling resistance, near-zero resistance, interference resistance, brake candidate, the adhesion wheel-slip substitution, the configured multiplier, and direct resistance. The root `Consist` particle path applies signed force and friction per particle, then aggregates mass, pressure, linkages, slack, and push/pull state over discrete steps. The Lean `ParticleState`, `ConsistState`, `irParticleStep`, and `consistStep` expose this boundary; the full Java linkage/collision implementation remains a refinement obligation.

8 Combined command and plant transition

The controller command is

$$u = (u_{throttle}, u_{train\ brake}, u_{independent\ brake}, u_{emergency}).$$

The stopping command deliberately allows a positive throttle cap in the model; it is not a hidden zero-throttle assumption. Its upper positive traction is included in the stopping bound. The transition is:

$$\begin{aligned} \text{spline localization} \rightarrow x, v_s \rightarrow (m, L, \kappa, \text{slack}, \text{push/pull}) \rightarrow u_{throttle} \rightarrow (u_{train\ brake}, u_{independent\ brake}) \\ \rightarrow F_{net} \rightarrow \text{particle step} \rightarrow \text{next controller state.} \end{aligned}$$

`controllerToCombinedForce` (rendered as `controller_to_combined_force`) expands the five signed force branches under the command update. `plantTransition` then writes the same command-modified physics record into the next particle state. The `plant_transition_connects_command_and_force` theorem and `plant_braking_upper` connect command, force, acceleration, and discrete velocity. Saturation is represented by `clamp`; slew limits, pressure propagation, and emergency-channel timing are explicit runtime parameters still required from Lua.

9 Uniform guard and safety envelope

Let B_{min} be a lower brake-force bound valid for every particle and every branch over the complete observation-plus-actuation-plus-pressure delay horizon. Let T_{max} bound positive traction and A_{max} bound adverse grade, curvature, slack, push/pull, direct-resistance uncertainty, and localization uncertainty in force units. The uniform positive deceleration contract is

$$a_{min} = \frac{B_{min} - T_{max} - A_{max}}{m_{current}} > 0.$$

For measured distance d , speed v , total delay τ , geometry error e_g , and terminal tolerance e_t , the terminal-entry guard is

$$d > 0, \quad v > 0, \quad B_{min} > T_{max} + A_{max}, \quad d \geq v\tau + A_{max}\tau^2 + \frac{v^2}{2a_{min}} + e_g + e_t.$$

The first term is observation/command/pressure delay distance, the second is adverse delayed motion, the third is the certified stopping distance, and the last two are geometry and terminal budgets. The guard is evaluated at the current measured speed; there is no separate universal operational speed theorem.

Lean proves guard positivity, the discrete stopping envelope, and one-step no-crossing under explicit hypotheses. The envelope is

$$0 \leq d, \quad 0 \leq v, \quad v^2 \leq 2a_{min}d.$$

It is not enough to prove no crossing. Arrival is a separate runtime progress and stable-sample contract. The mathematical terminal state is $v_s = 0$; runtime acceptance additionally requires

$$|v_s| \leq \varepsilon_v, \quad |x| \leq \varepsilon_{stop}, \quad \text{stable for } N_{stable} \text{ observations.}$$

The position tolerance must satisfy

$$\varepsilon_{stop} \geq \varepsilon_{trace} + \varepsilon_{fit} + \varepsilon_{coupler} + \varepsilon_{sample-age} + \varepsilon_{command-delay} + \varepsilon_{numeric}.$$

No decimal tolerance is accepted without a measured or formally bounded term.

10 Arrival, holding, and failure states

The proof separates the following claims:

1. stopping-envelope preservation;
2. no-crossing safety;
3. velocity convergence under the certified discrete brake model;

4. terminal arrival under a non-vacuous progress witness;
5. stable terminal regulation and holding;
6. active-route feasibility and replan behavior.

Lean defines terminal acceptance and progress predicates but does not claim a terminal-progress theorem from those predicates alone. This prevents the old vacuous liveness claim. The Lua implementation must provide the low-speed IR progress/holding refinement before any stronger readiness status is possible.

Fail-closed states are required for stale or invalid sample, invalid spline localization, wrong route orientation, route revision change, missing route, marker mismatch, unavailable radio/detector data, profile mismatch, invalid numeric value, pressure delay overrun, and inconsistent plant response. Each must emit an in-game message, stop the controller, and require player resolution.

11 Claim inventory

Claim	Classification	Current evidence
Bezier endpoint/reverse identities	proved by Lean	IRProof
Centre estimate residual implication	proved by Lean under residual certificate	TraceCentreSound ; offset and bound are model/runtime inputs
IR turn is cubic approximation	derived from bytecode	root CubicCurve.circle()
Crossing/overpass routing exclusion	proved by Lean under topology evidence	geometry evidence must be supplied
Directed marker mapping	proved by Lean	explicit tolerance and approach
Detector order and profile mismatch	proved by Lean under event/info contract	API event delivery is runtime
Material constants and default multiplier	derived from bytecode	root classes and configuration
Signed force ledger	proved by Lean/derived from bytecode	full Java branch refinement outstanding
Combined throttle/brake transition	proved by Lean at contract level	Lua command policy and delay refinement outstanding
Guard positivity and one-step envelope	proved by Lean	lower-bound premises remain runtime obligations
Uniform IR robust arrival	unsupported/withdrawn	requires full multi-particle, delay, float, and Lua refinement
Formal optimality	unsupported/withdrawn	no independent efficiency objective proved

12 Implementation funnel

The future Lua implementation must add, in order:

1. an immutable trace record and compact-tracer centre estimator with measured residual bound;
2. cubic spline storage, arc marks, reverse orientation, incremental trace matching, topology ambiguity states, and persistent IDs;
3. graph rebuild from validated spline intervals and directed marker mappings;
4. detector event catalogue with immediate **info()**, order, UUID, and immutable profile hash;
5. front-coupler localization from stock definition/bogey/coupler offsets, never raw entity position;
6. route-local validation and revision-triggered replan/fail-closed behavior;
7. command allocation that sends throttle, train brake, independent brake, and emergency state together, with delay/pressure-age accounting;

8. a measured lower-force/upper-traction certificate for every profiled stock and curved/slack/push-pull consist;
9. terminal speed/position deadbands derived from the complete error sum, plus stable-observation holding;
10. adversarial and mutation tests before user approval.

Production Lua is intentionally unchanged by this artifact. The exact API calls, state fields, failure messages, and acceptance tests are listed in `pid-implementation-plan.md`.

13 Readiness

Gate A's Lean portion passes, but the complete Gates A–I do not. In particular, the Java-to-Lua floating-point conversion, complete IR particle linkage/collision refinement, route-local live graph mutation, detector timing, terminal holding, and robust arrival vector suite are not yet certified by the formal model. Therefore the only honest status is

NOT_READY

.

The final readiness report names each outstanding gate and does not convert a passing Lean build into implementation approval.